SPM Modular Project

Module 4:Partitioned Hash Join with Duplicates with MPI

Student: Karol Jinet Cardona Bolanos

Mat. 702023

Academic Year 2025/2026

1 Design Choices

The partitioned hash join is *partition-local*: the mapping function is a pure function of the key, so two records with equal keys always receive the same partition id and can only match inside that partition. We exploit this by making the **partition the unit of distribution**. If every record of R and S that maps to a given partition is delivered to a single process, that process runs the ordinary sequential local join on its data, and the global result is the sum of the local results. This is a distributed *Map+Reduce* pattern: scatter the data by key, map each owned partition to a local join, and reduce the partial results into the global join count and checksums.

Reuse of the Module 2–3 code: The MPI program is a single SPMD source that `#include`s the shared `common.hpp` *unchanged*. The partition mapping (`multiply_shift`), the build-phase table (`FlatCountMap`), the per-partition build/probe (`join_one_partition`), the two checksums, and the uniform/skewed generators are identical to the sequential baseline. The MPI layer adds only three things: distributed input generation, the redistribution, and the global reduction. We keep $P = 256$, the value used throughout Modules 1–3, so the comparison across modules is fair.

The four phases: The program is organized as a four-phase pipeline, executed in lockstep by every rank:

- **(0) Distributed generation:** Each rank produces its own disjoint, contiguous slice of R and S locally. The challenge is that the pseudo-random number generator (PRNG,`splitmix64`) is *stateful*: each value normally depends on the one before it, so to obtain record i one would have to draw all the records before it, which would force every rank to regenerate the slices of all lower ranks. We avoid this because each record consumes a *fixed* number of PRNG draws: one in the uniform case (the key value), and two in the skewed case: a first draw decides whether the record falls into a hot partition or not, and a second draw selects the actual key (either from the hot pool or as a uniform value). Crucially, both draws are consumed in *both* branches of that decision, so the stride is always two regardless of the outcome and never depends on the data. With this fixed stride, record i occupies known draw positions and is therefore *positionally addressable*: a rank computes the draws for its index range directly, reconstructing exactly the keys the sequential generator would have produced there. For the skewed workload the hot-key pool is rebuilt from a fixed seed, so every rank obtains an identical pool independently; the hot partitions remain the fixed set $\{0, \ldots, H-1\}$ inherited from Module 3.
- **(1) Redistribution:** After phase 0 the partitions are scattered: each rank holds records belonging to many different partitions. This phase gathers every partition onto a single owner. Each rank first maps each of its keys to a partition id and then to an owner rank, $owner(pid) = pid \bmod nprocs$. This cyclic rule (partition 0 to rank 0, 1 to rank 1, wrapping around) needs no shared state and works for any process count. Each rank then groups its records into one send block per destination rank, reusing the Module 2 partitioning machinery (histogram of the counts, prefix sum for the offsets, scatter into a contiguous send buffer). The exchange itself is a single `MPI_Alltoallv` per relation, where every rank sends a different block to every other rank and receives the blocks destined to itself. Since records are bare keys, the data travels as `MPI_UINT64_T` with no custom datatype. In `MPI_Alltoallv` each receiver must know in advance how many elements it will get from each sender in order to allocate its receive buffer, so a small `MPI_Alltoall` exchanging just the per-destination counts precedes the bulk transfer.

- **(2) Local join:** Each rank now owns complete partitions, so it runs the local join on its data with no communication.
- **(3) Reduction:** The three scalars (join count and the two checksums) are summed across ranks with a single `MPI_Reduce(MPI_SUM)`.

Choice of communicator: All communication uses the default communicator (`MPI_COMM_WORLD`), which spans every process. This is the natural choice because both collectives in the algorithm are inherently *global*: since a key may hash to any partition, and therefore to any owner rank, the redistribution is an all-to-all in which every rank may send to and receive from every other, and the final `MPI_Reduce` likewise sums across all ranks. The communication scope thus coincides exactly with the set of all processes, so no sub-communicator is needed. This is also tied to running one rank per node: the entire redistribution crosses the inter-node network, which isolates the communication cost the module is meant to study. A hierarchical scheme with several ranks per node would instead hide part of that cost inside the nodes, mixing intra-node (shared-memory) and inter-node (network) communication into a single measurement and making the analysis harder to interpret. Defining process groups (`MPI_Comm_split`) would only add setup overhead and extra collectives without reducing this global fan-out.

2 Correctness

The join produces three scalars, `join_count` and two checksums, all of which depend only on the pairs of equal keys that match. A key-based partitioning merely relocates records and never creates or destroys a match, so these three values are **invariant to the number of partitions P, to the number of ranks, and to the partition function**: the distributed result must equal the sequential baseline *exactly*. Correctness is checked with the same two-level strategy as the previous modules. For small inputs, the `make check` target runs the MPI binary at `np`= 2 and 4 with the `-check` flag — on uniform data (N_R=10 000, N_S=15 000, P=64) and skewed data (N_R=20 000, N_S=30 000, *skew*=0.9, *hot*=4). Rank 0 independently regenerates the inputs and recomputes the result with a single-process reference, then asserts that `join_count`, `checksum1`, and `checksum2` match the distributed output; both `np` values yield identical scalars (e.g. `join_count` = 299 441 uniform, 4 090 370 skewed), confirming independence from the rank count. For large inputs, the same three values are compared between `hashjoin_seq` and `hashjoin_mpi` with identical parameters, and are bit-exact. All checks pass at every rank count $1, 2, 4, 8$, for all three redistribution modes and the hybrid build: the full uniform workload gives `join_count` = 4 000 007 100 (matching Module 2–3) and the skewed workload 994 181 140 057 (matching Module 3).

3 Experimental Setup and Reproducibility

All runs use the `spmcluster` compute nodes (Open MPI 5.0.3) connected by 10 Gigabit Ethernet, launched with `srun --mpi=pmix`. Nodes are allocated `--exclusive` with one MPI rank per node, so each run owns whole machines and all redistribution traffic crosses the inter-node network — the trade-off this module is meant to expose. The strong-scaling workload matches Modules 2–3: N_R = 50 M, N_S = 80 M, P = 256, seed 13, `max-key`= 10^6; reported times exclude input generation (T_{seq} = 3.43 s uniform, 2.73 s skewed).Weak scaling uses the same per-rank base of 10 M/16 M records as Modules 2–3. Spot re-runs on the exclusively-allocated nodes varied within a few percent, so single measurements are reported.

For the pure-MPI runs no NUMA pinning is needed: a single, single-threaded process on an exclusive node has no intra-node thread scheduling to constrain. For the hybrid runs the OpenMP threads are bound to physical cores with `OMP_PROC_BIND=close` and `OMP_PLACES=cores`, while `srun --cpu-bind=none` hands affinity control to the OpenMP runtime so the two mechanisms do not fight over the cores.

4 Uniform Workload

Table 1: Uniform strong scaling (blocking redistribution). Speedup and parallel efficiency are relative to the sequential baseline 3.43 s.

nodes	t_{redist} (s)	t_{join} (s)	t_{total} (s)	speedup	efficiency
1	2.263	3.496	5.759	0.60×	0.60
2	1.308	1.696	3.004	1.14×	0.57
4	0.727	0.829	1.556	2.21×	0.55
8	0.389	0.239	0.628	5.47×	0.68

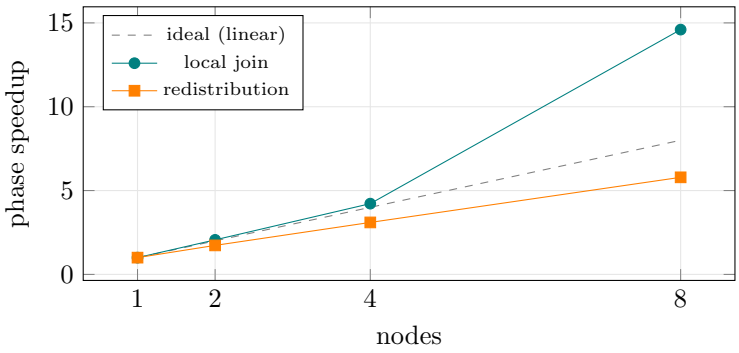


Figure 1: Speedup of the two phases vs. the ideal line. The local join lies *above* linear (super-linear), while redistribution lies *below* (sub-linear).

The final reduction is omitted from the breakdown: it sums only three 64-bit scalars across the ranks and costs under 0.2 ms at every scale, so $t_{\text{total}} \approx t_{\text{redist}} + t_{\text{join}}$. The two phases behave very differently as nodes are added:

- **One node is slower than sequential** (0.60×). Even with a single process the MPI path performs a redistribution to itself *and* a second partitioning pass inside the local join, so its t_{join} is essentially the whole sequential partition+join: a single rank pays for two passes over the data (the redistribution bucketing plus the local re-partitioning) where the sequential baseline does only one. This overhead is fixed, so it is only amortized once the work is split across two or more nodes; from two nodes onward the distributed version starts to pay off.
- **The local join scales super-linearly.** Its time falls 3.50 → 1.70 → 0.83 → 0.24 s: from four to eight nodes it drops by about 3.5× while the node count only doubles, so the speedup grows faster than the number of nodes, reaching 14.6× at eight nodes (3.50/0.24). The cause is cache behaviour: as each rank owns fewer records, its per-partition build tables shrink until they fit a faster level of the cache hierarchy, making each lookup cheaper.
- **Redistribution scales sub-linearly** (2.26 → 1.31 → 0.73 → 0.39 s, a 5.79× speedup at eight nodes). It does get faster as nodes are added, but not at the ideal rate (5.79× rather than 8×), because only its local bucketing part scales cleanly; the network all-to-all does not, since adding nodes neither raises the per-link bandwidth nor removes the contention of many simultaneous messages on the shared switch. Figure 1 shows the two phases against the ideal line: the join above it, the redistribution below.

At low node counts the join dominates the runtime (at one node it is 3.50 s against 2.26 s of redistribution), but because the join shrinks super-linearly while redistribution does not, the balance flips: by eight nodes the join has fallen to 0.24 s while redistribution is still 0.39 s. Redistribution is now the larger of the two phases, so further scaling is governed mainly by communication rather than by compute — the expected behaviour of a distributed all-to-all join, where the per-rank work keeps shrinking but the network does not. This also explains the efficiency column: it falls from 0.60 to 0.55 through four nodes, as usual, but then *rises* to 0.68 at eight nodes, against the normal trend, because the super-linear speedup of the join more than offsets the slower redistribution.

Table 2: Uniform weak scaling (per-rank 10M/16M, blocking), median of 3 runs. Efficiency is t_1/t_N.

nodes	t_{redist} (s)	t_{join} (s)	t_{total} (s)	efficiency
1	0.455	0.676	1.131	1.00
2	0.536	0.668	1.202	0.94
4	0.584	0.659	1.249	0.91
8	0.600	0.403	1.003	1.13

Weak scaling keeps the per-rank size fixed and lets the total grow with the node count. Efficiency stays at or above 0.91; the local-join phase is flat through four nodes and the eight-node sample even came in faster (0.40 s), pushing the total below the single-node time. To check that this super-unit efficiency was not a measurement artefact, this experiment was repeated three times and the table reports the median: the effect is stable across all three runs, so it is genuine, not noise. Its cause is the local-join phase, which runs faster at eight nodes (0.40 s vs. $\sim$ 0.66 s at fewer nodes): with more nodes each rank owns fewer but larger partitions ($256/nprocs$), so the join loops over fewer, more contiguous hash tables with less per-partition overhead. The essential point is that per-node throughput is preserved as the cluster grows, far better than the shared-memory weak scaling of Modules 2–3, which fell to 0.27 at $T = 16$ because all threads there contended for one node's DRAM bandwidth, whereas here each node adds its own.

5 Skewed Workload ($skew = 0.9$, $hot = 4$)

The skewed sequential baseline (2.73 s) is lower than the uniform one (3.43 s), so all skewed speedups are taken against this smaller reference. We focus here on how the distributed join behaves under skew.

Table 3: Skewed workload ($skew = 0.9$, $hot = 4$): strong scaling (total s and speedup vs. sequential 2.73 s) and weak scaling (per-rank 10M/16M, efficiency t_1/t_N).

	Strong scaling				**Weak scaling**			
nodes	t_{redist}	t_{join}	t_{total}	speedup	t_{redist}	t_{join}	t_{total}	eff.
1	2.261	2.749	5.011	0.55×	0.454	0.556	1.010	1.00
2	1.307	1.366	2.672	1.02×	0.570	0.556	1.126	0.90
4	0.718	0.709	1.427	1.92×	0.597	0.597	1.195	0.85
8	0.498	0.663	1.161	2.35×	0.836	1.146	1.982	0.51

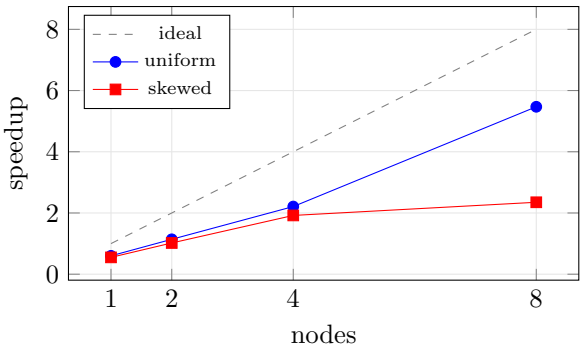


Figure 2: Strong-scaling speedup against the ideal line.

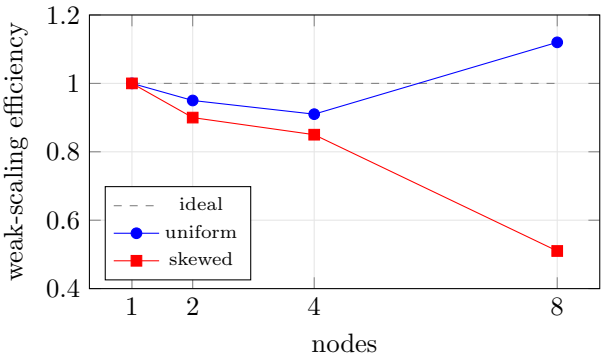


Figure 3: Weak-scaling efficiency.

The speedup *plateaus* at 2.35×, well below the 5.47× of the uniform case (Figure 2). The cause is visible in the join column, which barely moves from four to eight nodes (0.709 → 0.663 s). With $owner(pid) = pid \bmod nprocs$ the four hot partitions land only on ranks 0–3; beyond four nodes the extra ranks receive no hot partition, so the busiest rank's load does not shrink. With only H=4 hot partitions there is simply no way to spread that

work across more than four ranks, so adding the fifth through eighth nodes leaves the dominant cost untouched. Redistribution, by contrast, still scales normally, because its cost is uniform per record regardless of the key distribution.

Weak scaling makes the point sharper: efficiency *collapses* to 0.51 at eight nodes, because the hot-partition load on ranks 0–3 grows with the total problem size but is never distributed across the other ranks.

6 Blocking vs. Non-blocking Redistribution

The `-nb` flag selects the blocking exchange or one of two forms of overlap:

- **nb=0 – blocking.** One `MPI_Alltoallv` per relation.
- **nb=1 – communication/communication overlap.** R and S are issued together with `MPI_Ialltoallv` and completed with one `MPI_Waitall`, so the two transfers may proceed concurrently.
- **nb=2 – communication/computation overlap.** Both are posted; we wait only on R and then, *while S is still in flight*, partition R and build its per-partition hash tables (work that depends only on R), with periodic `MPI_Test` to nudge progress; finally we wait on S and probe. This is the textbook use of non-blocking communication: hiding a transfer behind independent computation.

Since nb=2 fuses redistribution and join, the comparable metric is the total time.

nodes	nb=0	nb=1	nb=2
1	5.759	5.770	6.454
2	3.004	2.995	3.769
4	1.556	1.557	1.984
8	0.628	0.618	0.802

Table 4: Uniform total time (s) by redistribution mode.

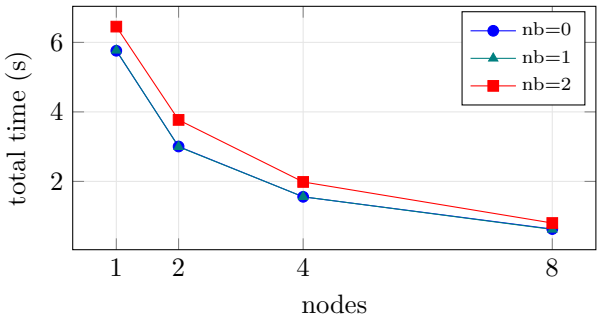


Figure 4: The three redistribution modes on the uniform workload.

nb=1 is within ±2% of blocking: the two posted exchanges share the same link, so overlapping them with each other buys essentially nothing. nb=2 is consistently *worse*, by +12% to +28%. To understand why, note that nb=2 changes *how the local join is organised*, not just when communication happens. The blocking version processes one partition at a time — it builds a small hash table for that partition, probes it, discards it, and moves to the next. These tables are small enough to stay in cache, so the join streams through them cheaply. To overlap the build of R with the still-incoming transfer of S, nb=2 must instead build *all* P hash tables up front and keep them alive at once (∼1.6 GB at one node). This breaks the cache-friendly streaming: the working set no longer fits in cache, every lookup reaches main memory, and the local join itself becomes slower.

The overlap nb=2 tries to exploit does not compensate for this, because on this cluster the transfer does not actually progress during the computation: there is no RDMA hardware, and this build of Open MPI reports `OMPI progress: no`, so a posted `Ialltoallv` advances mainly when the program is inside an MPI call, not while the CPU is busy building tables. We tested this directly by forcing an asynchronous progress thread (`UCX_PROGRESS_MODE=thread`): the nb=2 time was unchanged (3.80 s vs. 3.77 s at two nodes). Had the slowdown been caused by the missing overlap, enabling progress would have recovered it; it did not, which confirms that the real cost is the up-front materialisation of all hash tables, not the absence of overlap. Non-blocking communication therefore helps only when genuine asynchronous progress *and* non-degrading independent work are both available — neither holds here, an informative negative result.

7 Hybrid MPI + OpenMP

The hybrid build keeps one rank per node and adds 16 OpenMP threads per rank, threading the loop over owned partitions inside the local join with `schedule(dynamic,1)`.

Table 5: Total time (s): pure MPI (1 core/node) vs. hybrid (16 threads/node).

	uniform		skewed	
nodes	pure	hybrid	pure	hybrid
1	5.759	4.977	5.011	4.144
2	3.004	2.698	2.672	2.621
4	1.556	1.370	1.427	1.386
8	0.628	0.524	1.161	1.150

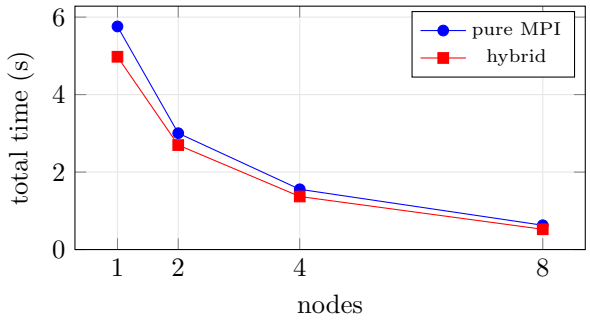


Figure 5: Uniform workload: the hybrid is faster at every node count.

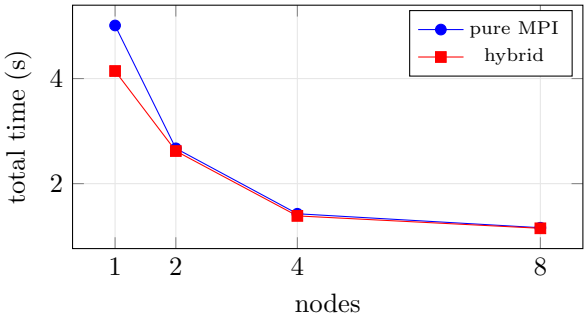


Figure 6: Skewed workload: the gain is largest at one node and nearly converges to the pure version.

Uniform. With the threads bound to cores, the hybrid is faster than the pure version at *every* node count, by 14–17%. The reason is that OpenMP accelerates the local join: with 16 threads sharing the partition loop, the join time drops at every scale (for example 3.50 → 2.71 s at one node, and 0.24 → 0.14 s at eight). The gain, however, is far from the 16× one might expect from 16 threads. This is an Amdahl effect: only the loop over partitions is parallelised, while the rest of each rank's work, the redistribution and the local re-partitioning of the received data, stays serial. Those serial phases put a ceiling on what threading can recover, so at one node 16 threads cut the total time by only about 1.16×. At eight nodes the join is already tiny, so the absolute saving is just 0.10 s, but the hybrid still comes out ahead.

Skewed. Here the benefit depends strongly on the node count, and the reason ties back to where the hot partitions live. The hybrid helps most at *one* node (−17%): with a single rank, all four hot partitions sit on the same machine, and `schedule(dynamic,1)` lets the 16 threads pick them up in parallel, one heavy partition per thread, which is exactly the within-node rebalancing studied in Module 3. As nodes are added, MPI spreads the four hot partitions across separate ranks (at most one hot partition per node once there are four or more nodes). At that point each node holds a single hot partition, which is one iteration of the loop and cannot be split across threads, so OpenMP has nothing left to rebalance and the hybrid time converges to the pure one. The two levels of parallelism are therefore complementary: OpenMP rebalances hot partitions *within* a node, while MPI distributes them *across* nodes, but neither can spread the hot work beyond the $H = 4$ partitions that carry it.

8 Comparison and Conclusions

The distributed implementation produces output identical to the sequential baseline at every rank count and in every redistribution mode, so the parallelism is correct by construction. On performance, it clearly surpasses the shared-memory versions of the previous modules. On a single node those reached 0.71 s at $T = 16$ ($\approx 4.9\times$),

plateauing against the DRAM bandwidth of one NUMA node, with a weak-scaling efficiency of only 0.27. The distributed version instead reaches 0.628 s at eight nodes with pure MPI (5.47×, using a single core per node) and 0.524 s with the hybrid build (6.56×): both beat the best single-node result, and with far fewer cores, because each node contributes its own memory bandwidth rather than contending for one node's.

Performance on the uniform workload is governed by the balance between two phases that scale in opposite ways: the local join, which is super-linear as each rank's working set shrinks into cache, and the all-to-all redistribution, which is sub-linear because the network bandwidth is fixed. As nodes are added the redistribution overtakes the join and becomes the larger of the two phases, so further scaling is paced by data movement rather than by compute; non-blocking communication does not change this, since the cluster offers no asynchronous transfer progress and the overlap would only break the cache-friendly join. Under skew the speedup is instead limited by load imbalance: with $H = 4$ hot partitions, MPI cannot spread the heavy work across more than four ranks, so the strong-scaling speedup plateaus and weak-scaling efficiency collapses. The hybrid build recovers part of this loss, but only while several hot partitions still co-reside on a node, where OpenMP can rebalance them across threads. The recurring lesson across both workloads is that the join itself parallelises almost for free; what paces the distributed performance is data movement under uniform load and load imbalance under skew.

9 Limitation and Possible Extension

The main limitation of the design appears under skew. Because the owner mapping is static, every partition has exactly one owner rank. The heavy work is concentrated in the H hot partitions, so it can be shared by at most H ranks no matter how many nodes are available: beyond H nodes the extra ranks receive only cold partitions and sit idle. This is the plateau seen in the skewed strong scaling, where the speedup stops improving past four nodes.

The placement of the hot partitions decides how well that ceiling is reached. The adjacent set $\{0, 1, 2, 3\}$ used here is the favourable case: under $\mathit{owner}(\mathit{pid}) = \mathit{pid} \bmod \mathit{nprocs}$ the four hot partitions fall on four *distinct* ranks, which is the best possible balance for $H = 4$. A less favourable placement — for instance hot indices that are all congruent modulo *nprocs* — would instead send several hot partitions to the same rank, making things worse. In every case, though, the ceiling of H active ranks remains.

Removing this ceiling would require letting more than one rank work on the same hot partition. To do that, each of those ranks needs its own copy of that partition's build-side hash table, so the build side of the hot partitions would have to be *replicated* (only the few hot partitions, not all P). The probe side could then be divided among them. The price is a more complex reduction: a hot partition processed by several ranks produces partial counts and checksums that must be carefully recombined into the correct totals, instead of the single global sum the current design uses.